Assignment 2 — Distributed Banking Platform

Sharding, replication and two-phase commit | Technical report

Django/DRF bank shards · FastAPI shard-aware gateway · PostgreSQL 16 · Docker Compose

The system implements a two-shard banking platform in which a client talks only to one gateway. The gateway resolves every account through a range shard map, load-balances over healthy replicas of the owning shard, and runs cross-shard transfers through a two-phase-commit (2PC) coordinator whose durable decisions live in an independent database. Same-shard transfers stay inside one PostgreSQL transaction. The report explains each requirement and maps it to the implementation and to the test evidence.

Result summary. Client acceptance suite: **16/16** requirements demonstrated. Non-interactive experiment harness (Experiments 5, 6 and 7): **36/36** checks passed. Concurrent test: 60 cross-shard and 40 same-shard transfers, all completed, money conserved, **0** unresolved coordinator rows and **0** prepared transactions left behind. The DRF browsable API is available on the replica ports (`8001`–`8004`) for inspection; the banking client still uses only `8000`.

1. Deployment at a glance

Component	Container / image	Responsibility and endpoint
Banking client	`client test/client.py`	Balance, deposit, withdraw, transfer. Sends HTTP only to the gateway; never resolves shards or replicas.
Gateway	`gateway` (FastAPI + Uvicorn)	Only client entry point. Shard selection, round-robin replica selection, health tracking, request validation, 2PC coordinator. `http://127.0.0.1:8000`.
Shard A	`api-a1`, `api-a2` (Django + Gunicorn)	Replicas of shard A; both share `db-a`. Inspection ports `8001` and `8002`.
Shard B	`api-b1`, `api-b2` (Django + Gunicorn)	Replicas of shard B; both share `db-b`. Inspection ports `8003` and `8004`.
Shard databases	`db-a`, `db-b` (PostgreSQL 16)	`banking_a` holds A0001–A1000; `banking_b` holds A1001–A2000. Each database is the single authoritative copy of its range.
Coordinator store	`coord-db` (PostgreSQL 16)	`banking_coord` holds the durable coordinator state machine and decisions, separate from both shards.
Configuration	`config/shard_map.yaml`, `config/gateway.yaml`	Account ranges, replica URLs, health thresholds and request timeouts, mounted read-only into the gateway.

PostgreSQL is started with `max_prepared_transactions=64` so that `PREPARE TRANSACTION` is available on both shards. The shard image collects DRF/admin static assets at build time and serves them with WhiteNoise; Gunicorn runs one `gthread` worker with 48 threads. The gateway disables Uvicorn access logging because it emits its own structured request logs and the health probe would otherwise dominate the log view.

2. Architecture and request path

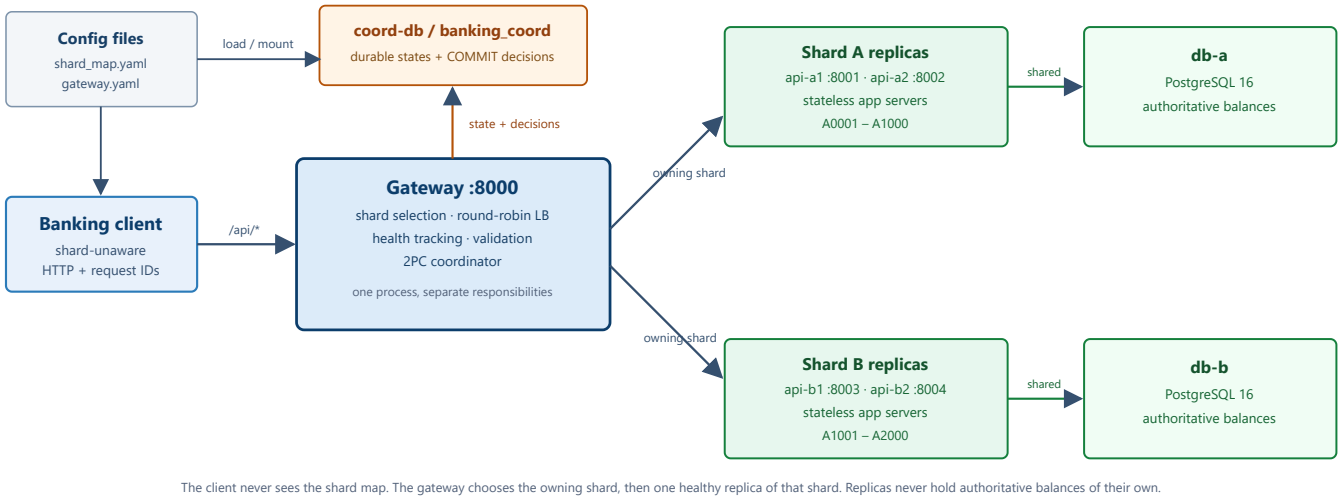


Figure 1 — Deployment and control/data paths.

Component responsibilities

- **Client** (`client test/client.py`): performs balance, deposit, withdraw and transfer calls against `BASE_ROOT=http://host:8000`. It does not import the shard map, does not query both shards, and cannot choose a replica. The transfer helper adds a unique `request_id` and an `X-Request-ID` header.
- **Gateway** (`gateway/main.py`, `balancer.py`, `coordinator.py`): validates transfer payloads (positive amount, at most two decimals, different accounts, request-ID syntax), resolves the shard, selects a healthy replica, proxies local operations, and hosts the 2PC coordinator. It is the only component that knows the topology.
- **Shard replicas** (`transactions/`): Django/DRF applications. They own the account and participant-commit models, expose the public account API and internal `/internal/2pc/*` endpoints, enforce ownership on the write path, and log the replica ID in every response header and structured log line.
- **Coordinator store**: a PostgreSQL database used only for the transaction state machine. It is deliberately not in either shard database, so a shard outage cannot erase a recorded decision.

3. Sharding, routing and load balancing

3.1 Deterministic range shard map

Partitioning is range-based. `config/shard_map.yaml` is the single topology source and states the ranges explicitly:

```
version: 1
epoch: 1
ranges:
  - start: A0001
    end: A1000
    shard_id: shard-a
  - start: A1001
    end: A2000
    shard_id: shard-b
```

Account identifiers have the form `A` plus four digits, so lexicographic comparison is also numeric comparison. The map loader validates the format, rejects inverted or overlapping ranges, and rejects gaps between adjacent ranges; the gateway's loader also requires every shard named by a range to have at least one replica. The same map is loaded by each shard at startup: `transactions/sharding.py` checks that this process's `SHARD_ID` owns at least one range and fails fast otherwise. This prevents a misconfigured replica from serving accounts it does not own.

Each account has exactly one authoritative shard. The full dataset is not copied to both shards: `banking_a` contains only A0001–A1000 and `banking_b` only A1001–A2000. A request for A0500 is always routed to shard A; A1500 is always routed to shard B. The gateway refuses account listings through `/api/accounts/` so that no code path accidentally performs a global scatter-gather balance query.

3.2 Routing and replica selection

1. The gateway reads the account number from the URL or transfer payload and calls `shard_for(account)`. If no range matches, it returns `404 UNKNOWN_ACCOUNT` without contacting any shard.
2. Within the owning shard, `select(shard_id)` walks the healthy replica list in round-robin order. An unhealthy replica is never selected while a healthy one exists; an optional exclusion set lets a retry avoid the replica that just failed.
3. For reads, the gateway may retry once on a transport error with the next healthy replica. For writes it does not blindly retry, because a lost response has an uncertain outcome. Transfers instead carry an idempotent `request_id`, and the status API can be polled.
4. Every response carries `X-Served-By`, `X-Shard` and `X-Request-ID`. The shard log records the replica, account, action, result, balance and latency; the gateway log records the chosen shard and replica.

3.3 Health detection and failover

Each replica exposes `GET /health`; the gateway probes all replicas every 2 seconds with a 1-second timeout. Two consecutive failures mark a replica unhealthy; three consecutive successes bring it back. A shard is considered available while at least one of its replicas is healthy. These thresholds are configuration values (`config/gateway.yaml`), not hard-coded in the request path.

Load-distribution check. Twelve sequential balance reads for A1001 (shard B) were served by `shard-b-1` 6 times and `shard-b-2` 6 times. Twelve reads for A0501 (shard A) were split 6/6 between `shard-a-1` and `shard-a-2`. No request for a shard-A account was served by a shard-B replica.

Failover check. With `api-b1` stopped, twelve reads for A1001 were all served by `shard-b-2`; shard-A traffic continued to be split across its two replicas. After `api-b1` was restarted it rejoined the pool.

4. Concurrency control and banking operations

4.1 Replica consistency

Replicas are stateless with respect to balances: they do not cache balances and they do not keep independent copies. Every read and write goes to the shard database, which is the authority. A write executes inside `transaction.atomic()` and begins with `SELECT ... FOR UPDATE` (`select_for_update()`), so concurrent operations through different replicas serialize on the same row. This prevents lost updates even when both replicas process requests for the same account at the same time. The database also enforces `CHECK (balance >= 0)`; the application checks the balance before a withdrawal so that the common case returns a useful `422` instead of a constraint error.

4.2 Local operations and same-shard transfers

Operation	Transaction scope	Guarantee
Balance inquiry	Single shard, read-only	Reads the authoritative row; no cross-shard aggregation.
Deposit / withdraw	One database transaction on the owning shard	Row lock, atomic update, non-negative balance constraint, durable commit before the response.
Same-shard transfer	One database transaction on the owning shard	Both rows are locked in ascending account order, then both balances are updated. Either both updates commit or neither does. No coordinator is involved.
Cross-shard transfer	Two participant transactions coordinated by 2PC	Each shard prepares its own row change; the durable decision is recorded before commit or abort is sent; see Section 5.

Sorting the two account numbers before locking is a small but important detail: two opposite-direction same-shard transfers cannot each hold one row and wait for the other. The same global ascending order is used by the coordinator when it prepares the two shards, so the cross-shard case has the same deadlock-avoidance property.

4.3 Participant transactions

A 2PC participant cannot use `transaction.atomic()` because Django commits on block exit, while the protocol needs a prepared-but-not-committed state. `transactions/participant.py` therefore opens a short-lived psycopg2 connection in autocommit mode, issues `BEGIN`, sets `lock_timeout` and `statement_timeout`, locks the account row `FOR UPDATE`, validates

funds and ownership, applies the tentative update, and executes `PREPARE TRANSACTION '<txid>:<shard>'`. PostgreSQL's `pg_prepared_xacts` is the authoritative PREPARED state. The participant does not resolve prepared transactions on startup; recovery belongs to the coordinator.

The prepare timeouts are 4 seconds for lock acquisition and 5 seconds for the whole statement, both below the coordinator's 6-second RPC timeout. A participant that cannot acquire a hot row in time votes NO with `LOCK_TIMEOUT` or `PREPARE_TIMEOUT`. This bound is part of the correctness argument: once the coordinator's RPC attempt has ended, no prepare can still be running and later create an orphaned prepared transaction.

4.4 Admission control and log hygiene

Each shard process enforces `MAX_CONCURRENT_CLIENTS=20` with a process-local counter in the logging middleware; excess requests receive `503` immediately. The middleware logs only `/api/` requests, so health probes and static files do not flood the transaction log. Gunicorn uses one `gthread` worker with 48 threads; the pool is deliberately larger than the admission limit. During testing, a 12-thread pool allowed blocked prepares on a hot account to occupy every thread, delaying the commit that would release the row lock and producing avoidable lock timeouts. With 48 threads the commit path always has capacity, and the concurrent experiment passes. The gateway also disables Uvicorn access logs; its structured request/coordinator logs are the useful operational record.

5. Two-phase commit

5.1 Protocol

The gateway determines the source and destination shards from the shard map. If they differ, it calls the coordinator with the accounts, amount, shards, txid and optional `request_id`. The coordinator inserts an `INITIATED` row, moves to `PREPARING`, and prepares one participant per shard.

- **Phase 1 — prepare.** Participants are sorted by account number and prepared in that global order. Each participant validates ownership, amount and funds, locks its account row, applies the tentative update, runs `PREPARE TRANSACTION`, and votes YES or NO. A NO or an indeterminate outcome leads to abort.
- **Durable decision.** If and only if every participant voted YES, the coordinator writes `COMMIT_DECIDED` with decision `COMMIT` to `coord-db` before sending any phase-2 message. Otherwise it writes `ABORT_DECIDED` with decision `ABORT`. The schema's check constraints make an inconsistent state/decision pair unrepresentable.
- **Phase 2 — commit or abort.** The coordinator sends `COMMIT PREPARED` or `ROLLBACK PREPARED` to every prepared participant; commits are issued concurrently. Participants are idempotent: a repeated COMMIT for an already committed GID is a no-op, and a `ParticipantCommit` marker prevents a later rollback from undoing a recorded commit.
- **Completion.** The transaction becomes `COMPLETED` only after every participant acknowledges. If an acknowledgement is missing, the row stays in `COMMIT_DECIDED` and recovery retries; the gateway never reports success early.

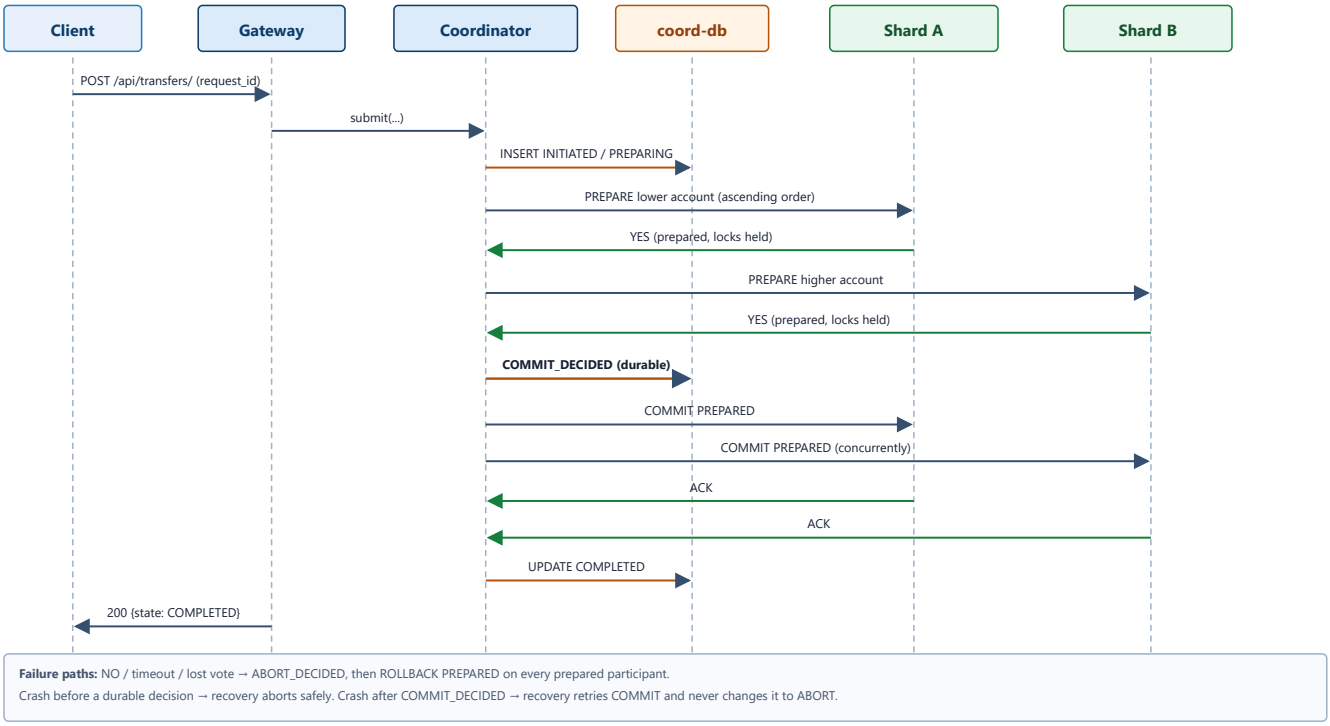


Figure 2 — Successful cross-shard transfer. Commits are issued concurrently; they are drawn sequentially for readability.

5.2 Idempotency and repeated messages

Every cross-shard transfer has a coordinator-generated 32-hex-character `txid`. If the client supplies a `request_id`, it is stored in a `UNIQUE` column. A retry with the same `request_id` and the same accounts/amount returns the existing transaction; the same `request_id` with different details returns `409 REQUEST_ID_CONFLICT`. On the participant side, `COMMIT PREPARED` is retried safely because the participant first checks `pg_prepared_xacts` and the `ParticipantCommit` marker. A rollback after a recorded commit is refused rather than silently destroying a committed effect.

6. Durable state and recovery

6.1 Persistent states

State	Durable meaning	Allowed next steps
`INITIATED`	Row created; no participant contacted	PREPARING, or abort if recovery finds a stale initiated transaction.
`PREPARING`	At least one PREPARE has been sent	PREPARED if all vote YES; ABORT_DECIDED on a NO or timeout; recovery may abort after the grace period.
`PREPARED`	All participants voted YES, but no decision is durable yet	COMMIT_DECIDED if the coordinator is still deciding; recovery aborts a fully prepared transaction that has no durable decision.
`COMMIT_DECIDED`	Durable COMMIT decision; phase 2 may be incomplete	COMPLETED only after every participant acknowledges. Recovery retries COMMIT; it never converts to ABORT.
`ABORT_DECIDED`	Durable ABORT decision	ROLLBACK PREPARED on all prepared participants, then ABORTED.
`COMPLETED` / `ABORTED`	Terminal state	No further state changes.

The coordinator table has a primary key on `txid`, a unique `request_id`, a partial index over non-terminal states, and check constraints that tie each state to its decision. The decision and the state are written in one row update, so there is no window in which the database contains a COMMIT decision with an ABORT state (or the reverse).

6.2 Recovery loop

The coordinator runs a recovery sweep every 5 seconds. It selects up to 50 non-terminal rows, takes a per-transaction lock so the request path and recovery cannot race, and calls the same state machine with `wait=False`. The handling depends

on the durable point reached:

- `INITIATED` or `PREPARING` older than the 12-second grace period → abort without preparing, recording `RECOVERED_INITIATED` or `RECOVERED_PREPARING`.
- `PREPARED` with no decision → abort, recording `RECOVERED_UNDECIDED`. A prepared transaction that never reached a durable COMMIT is not committed.
- `COMMIT_DECIDED` → resend COMMIT to every participant; mark COMPLETED when acknowledged. The decision is immutable.
- `ABORT_DECIDED` → resend ROLLBACK; mark ABORTED when acknowledged.

6.3 Required recovery scenarios

Case	Situation	Observed / implemented behavior
A	One shard votes YES, the other rejects (for example insufficient funds).	Coordinator records ABORT_DECIDED and sends ROLLBACK PREPARED to every participant that prepared. No balance changes. The client receives `422 ABORTED` and the txid can be queried.
B	All participants prepared; coordinator stops before acknowledging phase 2.	On restart, the unresolved row is read from `coord-db`. The committed test pauses after the durable decision, stops the gateway, verifies both prepared transactions survive, then restarts; recovery completes the transaction. If the crash had occurred before a decision, recovery would abort safely.
C	COMMIT is recorded, but not all participants acknowledged.	Recovery sees COMMIT_DECIDED and retries COMMIT. It never replaces the decision with ABORT because a participant is temporarily down. Repeated COMMIT is harmless; the participant commit marker and prepared-xact check make the effect exactly once.

7. Results and test evidence

The stack was exercised through the gateway only, exactly as the client uses it. The table lists the available evidence; all checks were run on the final Docker Compose deployment described above.

Test surface	Result	What was demonstrated
Client acceptance suite	16/16	Connection, normal operations, unknown account, insufficient funds, negative/invalid/missing fields, unsupported action, malformed request, ACID isolation/atomicity/durability, local transfer, cross-shard transfer, idempotency and abort.
Experiments 5 + 6 + 7	36/36	Non-interactive harness covering local/cross-shard transfers, interrupted distributed transaction, and concurrent cross-shard/local transfers.
Experiment 5	PASS	Local transfer reports `mode=local`; cross-shard reports `mode=cross-shard`, `COMPLETED`, and a queryable txid; reversing each transfer restores both balances.
Experiment 6	PASS	After the durable COMMIT decision, the gateway was stopped. The decision and both prepared transactions survived; after restart, recovery completed the transfer exactly once; reverse transfer restored balances.
Experiment 7	12/12	60 concurrent cross-shard transfers (20 workers) over overlapping accounts: all HTTP 200 / COMPLETED, money conserved, opposite directions net to zero, no unresolved coordinator rows, no prepared transactions left. 40 concurrent same-shard transfers also passed.
Load distribution	6/6 per replica	Twelve reads for one account in each shard were split evenly between the two replicas of that shard; the other shard was never used.
Replica failover	12/12	With one shard-B replica stopped, all reads for a shard-B account went to the remaining healthy replica; shard-A traffic was unaffected.
DRF browsable API	200	All four replica inspection ports return the DRF HTML UI (`/api/accounts/`) and `/static/rest_framework/css/bootstrap.min.css` returns 200 under Gunicorn. The client does not use these ports.

Observed concurrency numbers. The 60-transfer cross-shard experiment completed in approximately 4.4–4.9 seconds across runs on the final build. The same experiment was repeated after the thread-pool change and passed every time. The test harness also checks conservation of money and the absence of leftover prepared transactions, not just HTTP status codes.

8. Limitations and deliberate scope

- **Single gateway process.** The gateway is a single container and therefore a single point of failure. The assignment requires a shard-aware gateway, not a highly available gateway; the coordinator is logically separate but shares the process.
- **Blocking 2PC.** Prepared participants hold row locks until the decision is applied. A hot account can make other prepares wait; the bounded lock/statement timeouts convert that into a safe NO vote, and the client can retry with the same `request_id`. This is correct but not a high-throughput protocol.
- **Static topology.** The shard map is a mounted YAML file with `epoch: 1`. Changing ranges requires a configuration update and restart; there is no online re-sharding, change capture, epoch fencing or migration state machine. Bonus I is intentionally not implemented.
- **No database replication.** Each shard has one PostgreSQL instance; only the application replicas are replicated. Database replication and failover were explicitly not required.
- **No self-healing controller.** Replica restart is an operator action (`docker compose restart`); the gateway only detects health and routes around a failed replica. Bonus II is intentionally not implemented.
- **Demo security.** Credentials are local demo values; there is no authentication, authorization or TLS. The DRF browsable API is exposed on host ports for inspection only.
- **Operational limits.** `max_prepared_transactions=64` bounds concurrent prepared transactions; the prepared state is durable but not monitored by an alert. Write requests are not automatically retried after a transport failure because the outcome is uncertain; the transaction-status API is the recovery tool.

9. Requirement traceability matrix

#	Requirement	How it is handled / evidence	Status
1	Client talks only to the gateway and is shard-unaware	`client test/client.py` uses only `BASE_ROOT:8000`; no shard map import; transfer helper adds a request ID.	**Done**
2	Gateway selects the shard by account ownership	`GatewayConfig.shard_for()` compares the account against the range map; unknown accounts return 404 before any shard call.	**Done**
3	Replica selection among healthy replicas	Per-shard round-robin in `balancer.py`; observed 6/6 split on each shard, replica ID in `X-Served-By` and logs.	**Done**
4	Two shards, each with its own persistent database	`db-a/banking_a` and `db-b/banking_b` are separate PostgreSQL containers and volumes.	**Done**
5	At least two replicas per shard sharing that database	`api-a1/api-a2` share `db-a`; `api-b1/api-b2` share `db-b`; replicas keep no authoritative balances.	**Done**
6	Shard map file/metadata used for routing	`config/shard_map.yaml` version 1, epoch 1; validated for format, overlap, gaps and ownership at startup.	**Done**
7	Deterministic partitioning; one authoritative owner per account	Range map A0001–A1000 / A1001–A2000; `owns()` is also checked on the shard write path.	**Done**
8	Full dataset is not copied to every shard	Each database contains only its range; account listing through the gateway is rejected to avoid global scatter-gather.	**Done**
9	Replica consistency: no lost updates across replicas	Shared authoritative database, `transaction.atomic()`, `select_for_update()` row locks, DB check constraint on non-negative balances.	**Done**
10	Availability: detect failed replicas and route to healthy ones	2-second active probes; 2 failures eject, 3 successes re-admit; observed failover 12/12 to the healthy replica.	**Done**
11	Local balance/deposit/withdraw ACID and durability	Single-shard atomic transactions with row locks; client ACID tests pass; balances survive restart.	**Done**

#	Requirement	How it is handled / evidence	Status
12	Same-shard transfer is one atomic local transaction	`/internal/transfers/local/` locks both rows in ascending order and commits once; experiment 5 passes.	**Done**
13	Cross-shard transfer uses 2PC	Gateway coordinator prepares both shards, records a decision, then commits/aborts; experiment 5 cross-shard path passes.	**Done**
14	Prepare phase: validate, durably prepare, vote	Participant locks the row, checks funds/ownership, applies the tentative update, runs `PREPARE TRANSACTION`, votes YES/NO.	**Done**
15	Commit only when all vote YES; otherwise abort	`_prepare_all()` returns a NO/unknown and the coordinator records ABORT_DECIDED before rollback.	**Done**
16	Decision durably recorded before phase 2	`COMMIT_DECIDED` / `ABORT_DECIDED` is written to `coord-db` before COMMIT/ROLLBACK is sent.	**Done**
17	Participants handle repeated COMMIT/ABORT safely	`pg_prepared_xacts` plus the `ParticipantCommit` marker make repeated COMMIT a no-op and prevent rollback after commit.	**Done**
18	Unique transaction ID and persistent status	32-hex `txid`; states INITIATED, PREPARING, PREPARED, COMMIT_DECIDED, ABORT_DECIDED, COMPLETED, ABORTED in `coordinator_transactions`.	**Done**
19	Coordinator rebuilds unresolved transactions after restart	Recovery loop every 5 s reads non-terminal rows and advances them; experiment 6 stops and restarts the gateway.	**Done**
20	Do not report success before all participants confirm COMMIT	COMPLETED is written only after every commit acknowledgement; otherwise the row stays COMMIT_DECIDED and recovery retries.	**Done**
21	Unresolved transactions expose PENDING/UNKNOWN by txid	`GET /api/transactions/{txid}` returns PENDING for non-terminal, COMPLETED/ABORTED for terminal, UNKNOWN for an absent/invalid txid.	**Done**
22	Scenario A: one YES, one reject → safe abort	ABORT_DECIDED then ROLLBACK PREPARED on all prepared participants; balances unchanged (client abort test).	**Done**
23	Scenario B: all prepared, coordinator stops → recover	Experiment 6 verifies the durable decision and prepared transactions survive the stop and recovery completes after restart.	**Done**
24	Scenario C: COMMIT recorded, not all acked → complete, never abort	Recovery sees COMMIT_DECIDED and retries COMMIT; the decision is immutable and participant commits are idempotent.	**Done**
25	Experiments 1–7	Distribution/routing, load split, concurrent replica operations, failover, local/cross-shard transfers, interrupted transaction and concurrent transfers are covered by the client suite and the experiment harness.	**Done**
26	Bonus I: live re-sharding under failures	Not implemented; reported explicitly as out of scope. The static map and epoch are documented in Section 8.	**N/A**
27	Bonus II: autonomous self-healing	Not implemented; replica recovery is an operator action, and the gateway only performs health-based routing.	**N/A**
28	Deliverables	Source code, Compose deployment and this technical report are in the repository; the images build locally. Publishing the images to Docker Hub and recording the demo video are submission steps outside the running system.	**External**

The report intentionally stays within ten A4 pages. The implementation and test harness are the source of truth; commands and expected outputs are kept in the project's client and experiment harness so the results can be reproduced against a fresh Compose deployment.